

内存中消息通信——概况

内存中进程间通信机制(IPC)

一、共享内存

1.1、共享内存的创建

1.2、共享内存段(Shared Memory Segments)

1.3、初始化

1.4、gdb调试跟踪

1.5、动态共享内存(dsm)

核心功能

具体实例分析

二、同步

2.1、latch机制

2.2、自旋锁(spinLock)

2.3、轻量级锁(Lightweight Locks)

2.4、锁组

2.5、同步多进程并发的barrier机制

Barrier 结构

核心功能描述

三、信号(Signal)

2.1、原始信号量的PV操作

2.2、现代系统的两种信号量

2.2.1、System V IPC

2.2.2、POSIX IPC

2.3、信号量的演化

2.4、postgres中信号的使用

2.5、常见信号量

2.6、pmsignal的实现

2.6.1、Postmaster接收和发送信号概述

2.6.2、PMSignalData 结构体

2.6.3、主要功能

2.6.4、gdb调试跟踪

2.7procsignal实现

2.7.1、procsignal接收和发送信号概述

2.7.2、信号处理函数

2.7.3、主要功能

2.7.4、gdb调试跟踪

四、消息队列

3.1、消息

3.2、单读单写共享消息队列

3.2.1、shm_mq 结构:

3.2.2、shm_mq_handle 结构:

3.2.2、主要功能

五、无效消息(SI Message)

SI Message

SI Message定义函数

SI Message核心实现

六、并行查询和计算

并行查询

并行计算同步

测试案例

内存中进程间通信机制(IPC)

PostgreSQL中的IPC（Inter-Process Communication，进程间通信）主要采用共享内存的方式来实现。即在系统中开辟一片所有进程都可以读写的内存空间，并约定好进程读写这片内存的时机和方式。这样，进程之间就可以通过这片共享的内存来交换数据，从而达到通信的目的。

一、共享内存

在PostgreSQL中，共享内存是用来存储那些需要被数据库系统中的多个进程共享的数据。这包括了缓冲池、锁表、后台进程的状态信息等。通过共享内存，PostgreSQL能够有效地管理大量并发事务和操作，而不需要每个进程单独持有数据的副本，从而大大提高内存使用效率和数据访问速度。

1.1、共享内存的创建

这里简单说一下共享内存的创建。在(\src\backend\storage\ipc\shmem.c)中可以看到，关于共享内存的申请工作。首先通过InitShmemAccess函数，通过共享内存头部执政申明初始指针和末尾指针。

```
1 void
2 InitShmemAccess(void *seghdr)
3 {
4     PGShmemHeader *shmhdr = (PGShmemHeader *) seghdr;
5
6     ShmemSegHdr = shmhdr;
7     ShmemBase = (void *) shmhdr;
8     ShmemEnd = (char *) ShmemBase + shmhdr->totalsize;
9 }
```

接着初始化共享内存分配器，包括负责初始化共享内存分配所需的自旋锁（spinlock），并确保后续分配按照缓存行边界对齐，并初始化 ShmemIndex 相关指针，后续再申请轻量级锁(LWLocks)已完成对 ShmemIndex哈希表的申请。

```
1 void
2 InitShmemAllocation(void)
3 {
4     PGShmemHeader *shmhdr = ShmemSegHdr;
5     char *aligned;
6
7     Assert(shmhdr != NULL);
8
9     ShmemLock = (slock_t *) ShmemAllocUnlocked(sizeof(slock_t));
10
11     SpinLockInit(ShmemLock);
12
13     aligned = (char *)
14         (CACHELINEALIGN((((char *) shmhdr) + shmhdr->freeoffset)));
15     shmhdr->freeoffset = aligned - (char *) shmhdr;
16
17     /* ShmemIndex 需要LWLocks锁 */
18     shmhdr->index = NULL;
19     ShmemIndex = (HTAB *) NULL;
20 }
```

1.2、共享内存段(Shared Memory Segments)

PostgreSQL 使用的信号量常常与共享内存段 (Shared Memory Segments) 相关，它们一起支持 PostgreSQL 多个进程间的协调工作。

在\src\backend\storage\ipc\ipci.c中有关于共享内存段和信号量加和，共同申请共享内存的代码。其中信号量的申请包括用于进程间同步的信号量 (ProcGlobalSemas)、用于自旋锁的信号量 (SpinlockSemas)。

```
1  Size
2  CalculateShmemSize(int *num_semaphores)
3  {
4      Size      size;
5      int       numSemas;
6
7      /* Compute number of semaphores we'll need */
8      numSemas = ProcGlobalSemas();
9      numSemas += SpinlockSemas();
10
11     /* Return the number of semaphores if requested by the caller */
12     if (num_semaphores)
13         *num_semaphores = numSemas;
14
15     .....
16     /* 共享内存段的申请 */
17 }
```

将信号量与共享内存段一起申请的好处是，可以减少内存碎片，可以分配到连续的内存，方便信号量对各个内存段的信号管理。

1.3、初始化

与进程间通信相关的初始化工作也在ipci.c文件中，在初始化的过程中会创建和申请多个相关的子系统。与进程间通信相关的模块包括但不限于：

(1) 创建轻量级锁(LWLocks)

CreateLWLocks();

LWLocks 是 PostgreSQL 中非常重要的同步机制，用于保护共享数据结构的并发访问，避免数据竞争。

虽然 LWLocks 本身并不直接用于进程间通信，但它为其他 IPC 机制提供了基础保障，例如保护信号量、共享队列等数据结构的一致性。

(2) 创建共享内存索引(InitShmemIndex)

```
InitShmemIndex();
```

ShmemIndex 是一个哈希表，用于存储和查找 PostgreSQL 中所有共享数据结构的地址和信息。

它为各个模块提供了一种统一的机制来访问共享数据结构，是实现 IPC 的基础设施之一。

(3) 锁管理器(Lock Manager)

```
InitLocks();
```

锁管理器负责管理 PostgreSQL 中的各种锁，包括表级锁、行级锁等。

锁是数据库并发控制的核心机制，用于协调多个进程对共享数据的访问，防止数据不一致。

锁管理器的实现依赖于信号量和共享内存等 IPC 机制。

(4) 进程表 (Process Table)

```
InitProcGlobal();, CreateSharedProcArray();, CreateSharedBackendStatus();
```

PostgreSQL 使用进程表来跟踪所有活跃的后台进程 (backend process) 的状态信息，例如进程 ID、当前执行的查询、持有的锁等等。

进程表的信息会被所有后台进程共享和访问，因此也需要使用共享内存和同步机制来保证数据一致性。

(5) 共享失效消息队列 (Shared Invalidation Message Queue)

```
CreateSharedInvalidationState();
```

当一个进程修改了共享数据（例如系统缓存中的数据）时，它需要通知其他进程，以便它们能够及时更新自己的缓存，保证数据一致性。

PostgreSQL 使用共享失效消息队列来实现这种机制，这也是一种 IPC 方式。

(6) 进程间信号机制 (Interprocess Signaling)

```
PMSignalShmemInit();, ProcSignalShmemInit();
```

PostgreSQL 使用信号量来实现进程间的信号传递，例如通知某个进程执行特定的操作，或者通知某个事件的发生。

PMSignalShmemInit 和 ProcSignalShmemInit 函数用于初始化与进程信号相关的共享内存结构。

1.4、gdb调试跟踪

跟踪postgres启动时分配内存的过程，主要跟踪以下函数，其位置在(src\backend\storage\ipc)中：

```

1 void *
2 ShmemInitStruct(const char *name, Size size, bool *foundPtr)
3 {
4     ShmemIndexEnt *result;
5     void *structPtr;
6
7     LWLockAcquire(ShmemIndexLock, LW_EXCLUSIVE);
8
9     if (!ShmemIndex)
10    {
11        PGShmemHeader *shmemseghdr = ShmemSegHdr;
12
13        /* Must be trying to create/attach to ShmemIndex itself */
14        Assert(strcmp(name, "ShmemIndex") == 0);
15
16        if (IsUnderPostmaster)
17        {
18            /* Must be initializing a (non-standalone) backend */
19            Assert(shmemseghdr->index != NULL);
20            structPtr = shmemseghdr->index;
21            *foundPtr = true;
22        }
23        else
24        {
25            /*
26             * If the shmem index doesn't exist, we are bootstrapping: w
27             e must
28             * be trying to init the shmem index itself.
29             *
30             * Notice that the ShmemIndexLock is released before the shme
31             m
32             * index has been initialized. This should be OK because no
33             other
34             * process can be accessing shared memory yet.
35             */
36            Assert(shmemseghdr->index == NULL);
37            structPtr = ShmemAlloc(size);
38            shmemseghdr->index = structPtr;
39            *foundPtr = false;
40        }
41        LWLockRelease(ShmemIndexLock);
42        return structPtr;
43    }
44
45    /* look it up in the shmem index */

```

```

43     result = (ShmemIndexEnt *)
44         hash_search(ShmemIndex, name, HASH_ENTER_NULL, foundPtr);
45
46     if (!result)
47     {
48         LWLockRelease(ShmemIndexLock);
49         ereport(ERROR,
50             (errcode(ERRCODE_OUT_OF_MEMORY),
51              errmsg("could not create ShmemIndex entry for data struc
52 ture \"%s\"",
53                  name)));
54     }
55
56     if (*foundPtr)
57     {
58         /*
59          * Structure is in the shmem index so someone else has allocated
60          * it already. The size better be the same as the size we are tryin
61          * g to initialize to, or there is a name conflict (or worse).
62          */
63         if (result->size != size)
64         {
65             LWLockRelease(ShmemIndexLock);
66             ereport(ERROR,
67                 (errmsg("ShmemIndex entry size is wrong for data stru
68 cture"
69                     " \"%s\": expected %zu, actual %zu",
70                     name, size, result->size)));
71         }
72         structPtr = result->location;
73     }
74     else
75     {
76         Size      allocated_size;
77
78         /* It isn't in the table yet. allocate and initialize it */
79         structPtr = ShmemAllocRaw(size, &allocated_size);
80         if (structPtr == NULL)
81         {
82             /* out of memory; remove the failed ShmemIndex entry */
83             hash_search(ShmemIndex, name, HASH_REMOVE, NULL);
84             LWLockRelease(ShmemIndexLock);
85             ereport(ERROR,
86                 (errcode(ERRCODE_OUT_OF_MEMORY),
87                  errmsg("not enough shared memory for data structure"
88                      " \"%s\" (%zu bytes requested)",

```

```

87         name, size)));
88     }
89     result->size = size;
90     result->allocated_size = allocated_size;
91     result->location = structPtr;
92 }
93
94 LWLockRelease(ShmemIndexLock);
95
96 Assert(ShmemAddrIsValid(structPtr));
97
98 Assert(structPtr == (void *) CACHELINEALIGN(structPtr));
99
100 return structPtr;
101 }
102

```

如下所示，是跟踪代码具体过程

```

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./postgres...done.
(gdb) b ShmemInitStruct
Breakpoint 1 at 0x7e0e90: file shmem.c, line 401.
(gdb) run
Starting program: /data/postgres/base/16.2/bin/postgres
Missing separate debuginfos, use: yum debuginfo-install glibc-2.28-164.el8.x86_64
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".

Breakpoint 1, ShmemInitStruct (name=name@entry=0xa827c1 "ShmemIndex", size=2896, foundPtr=fou
  at shmem.c:401
401         LWLockAcquire(ShmemIndexLock, LW_EXCLUSIVE);
Missing separate debuginfos, use: yum debuginfo-install libblkid-2.32.1-28.el8.x86_64 libcap-
_64 libgcrypt-1.8.5-6.el8.x86_64 libgpg-error-1.31-1.el8.x86_64 libmount-2.32.1-28.el8.x86_64
1-28.el8.x86_64 lz4-libs-1.8.3-3.el8_4.x86_64 pcre2-10.32-2.el8.x86_64 systemd-libs-239-51.el

```

name 参数的值是 "ShmemIndex"，这是正在被初始化的共享内存结构的名称。size=2896: 共享内存结构的大小为 2896 字节。foundPtr=foundPtr@entry=0x7fffffe83f: foundPtr 参数的地址为 0x7fffffe83f，这是一个指针，用于记录共享内存结构是否已经存在。

继续运行直到查看到ShemeBase变量的值，它指向共享内存段的起始地址。查看调用栈堆，分析函数调用过程

```

(gdb) p ShmemBase
$1 = (void *) 0x7fffec0ef000
(gdb) bt
#0  ShmemInitHash (name=name@entry=0xa827c1 "ShmemIndex", init_size=init_size@entry=64, max_size=max_size@entry=64,
infoP=infoP@entry=0x7fffff870, hash_flags=2588, hash_flags@entry=24) at shmem.c:370
#1  0x00000000007e1153 in InitShmemIndex () at shmem.c:307
#2  0x00000000007d9381 in CreateSharedMemoryAndSemaphores () at ipci.c:234
#3  0x00000000007802bc in PostmasterMain (argc=argc@entry=1, argv=argv@entry=0xe69eb0) at postmaster.c:1072
#4  0x00000000004f27eb in main (argc=1, argv=0xe69eb0) at main.c:198
(gdb) info locals
found = false
location = 0x7fffec0f9980
(gdb) p ShmemEnd
$2 = (void *) 0x7ffff4f81000
(gdb)

```

这里通过ShmemBase和ShmemEnd可以计算出分配的共享内存段的大小。前的调用栈，从最底层的函数开始，依次向上层函数展示：ShmemInitHash: 这是 ShmemInitStruct 函数内部正在调用的函数，用于初始化一个共享内存哈希表。InitShmemIndex: ShmemInitHash 函数是被 InitShmemIndex 函数调用的，该函数用于初始化共享内存索引。CreateSharedMemoryAndSemaphores: InitShmemIndex 函数是被 CreateSharedMemoryAndSemaphores 函数调用的，该函数用于创建和初始化共享内存段以及信号量。PostmasterMain: CreateSharedMemoryAndSemaphores 函数是被 PostmasterMain 函数调用的，该函数是 PostgreSQL 守护进程 (Postmaster) 的主函数。main: PostmasterMain 函数是被 main 函数调用的，它是 PostgreSQL 服务器的入口函数。

调用关系如下：main -> PostmasterMain -> CreateSharedMemoryAndSemaphores -> InitShmemIndex -> ShmemInitHash -> ShmemInitStruct

1.5、动态共享内存(dsm)

DSM允许PostgreSQL的多个后端进程共享内存区域，这对于实现高效的并行计算和资源共享至关重要。在src\backend\storage\ipc\dsm.c中有所有关于Postgres中所有关于动态共享内存的实现。通过源码分析可以了解到，动态共享内存是众多共享内存相关插件的实现基础。开发者可以很好地使用动态共享内存来实现插件和扩展的开发。

动态共享内存段有以下几个特点：

1. 动态共享内存段需要进行引用计数，以便在删除最后一个映射后，该段会自动消失。
2. 如果backend进程被恶意终止，则postmaster进程需要在崩溃并重新启动的过程中删除所有剩余的段，就像它需要重新初始化主共享内存段一样。
3. 如果所有进程都被异常终止，那么下一次postmaster进程启动则需要清理仍然存在的所有段，就像对主共享内存段所做的一样。

核心功能

动态共享内存段 (DSM Segment)： 代表一个可以被多个进程共享的内存区域。每个DSM Segment由一个dsm_segment结构体表示，包含了段的句柄 (handle)、映射地址

(mapped_address)、映射大小 (mapped_size)、资源所有者、在控制段中的槽位 (control_slot) 等信息。

控制段 (Control Segment)：用于维护所有动态共享内存段的元数据，包括每个段的句柄、引用计数 (refcnt)、被映射的页数等。该段由dsm_control_header和一系列dsm_control_item组成，每个dsm_control_item记录了一个动态共享内存段的信息。

创建 (Create)、附加 (Attach)、和分离 (Detach)：提供了创建新的DSM Segment、附加到已经存在的DSM Segment以及从DSM Segment分离的函数。这些操作允许进程动态地管理共享内存资源。

具体实例分析

在dsm中普遍存储的有两种动态共享内存类型，一种是MMAP，用于映射共享内存地址；另一种就是POSIX或System V信号量。当dynamic_shared_memory_type为MMAP时

```
PL/SQL |
1  postgres=# alter system set dynamic_shared_memory_type=mmap;
2  ALTER SYSTEM
3
4  postgres=# select name, setting from pg_settings where name like 'dynamic_s
   hared_memory_type';
5           name                | setting
6  -----+-----
7  dynamic_shared_memory_type | mmap
8  (1 row)
```

可以看到postgres在目录下创建了两个mmap类型的动态共享内存控制段

```
[postgres@3a6fcfb8b59e data]$ cd ./pg_dynshmem/
[postgres@3a6fcfb8b59e pg_dynshmem]$ ls
mmap.1624067358  mmap.3845110148
[postgres@3a6fcfb8b59e pg_dynshmem]$ ls -lrt
total 1052
-rw----- 1 postgres dba    26976 Jun 21 07:13 mmap.3845110148
-rw----- 1 postgres dba 1048576 Jun 21 07:13 mmap.1624067358
[postgres@3a6fcfb8b59e pg_dynshmem]$
```

将把dynamic_shared_memory_type为posix时

```

1 postgres=# alter system set dynamic_shared_memory_type=posix;
2 ALTER SYSTEM
3
4
5 postgres=# select name, setting from pg_settings where name like 'dynamic_s
   shared_memory_type';
6          name          | setting
7 -----+-----
8 dynamic_shared_memory_type | posix
9 (1 row)

```

系统则将动态共享内存控制端存在了/dev/shm的系统目录下

```

[root@3a6fcfb8b59e /]# cd /dev/shm/
[root@3a6fcfb8b59e shm]# ls -l
total 1052
-rw----- 1 postgres dba 1048576 Jun 21 07:18 PostgreSQL.2249582440
-rw----- 1 postgres dba 26976 Jun 21 07:18 PostgreSQL.2302977158
[root@3a6fcfb8b59e shm]#

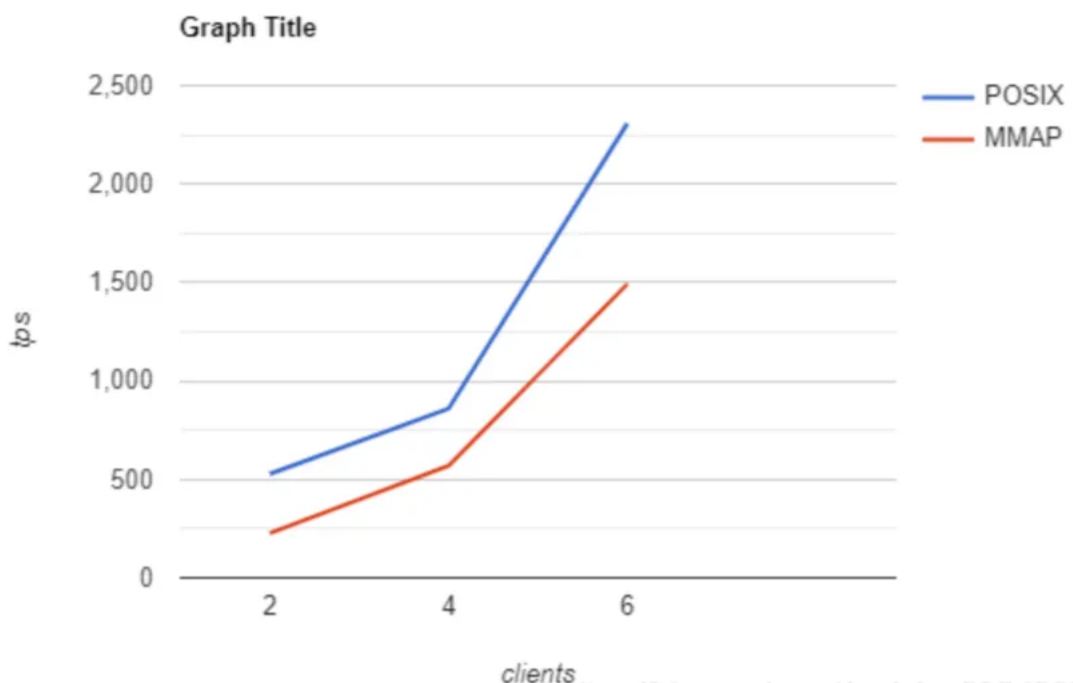
```

使用如下shell脚本对两种不同的数据类型进行测试

```

1 #!/bin/sh
2 for clients in 2 4 6 8 10
3 do
4   THREADS=${clients}
5   /data/postgres/base/16.2/bin/pgbench -h 127.0.0.1 -j ${THREADS} -c ${client
   s} -T 20 -S postgres
6   > result_mmap${clients}.txt
7 done

```



二、同步

2.1、latch机制

在\src\backend\storage\ipc\latch.c文件中有具体latch的实现。代码开头部分就详细解释了Latch机制的基本原理，以及它如何克服poll()和信号处理函数中存在的竞争条件。

1. 自管道技巧 (Self-pipe Trick) 与 poll()

在使用 poll() 实现时，为了克服 poll() 与在信号处理器中设置全局标志间的竞态条件，采用了所谓的自管道技巧 (self-pipe trick)。竞态条件是指程序执行结果的不确定性，它取决于事件或进程的执行顺序。

当一个 latch 被设置，且当前进程正在等待它时，信号处理程序通过向一个管道写入一个字节来唤醒 WaitLatch 中的 poll()。这是因为信号本身并不会在所有平台上中断 poll()；即使在那些会中断的平台上，若信号刚好在调用 poll() 之前到达，也不会阻止 poll() 进入休眠状态。然而，管道上的一个传入字节可以可靠地中断休眠状态，并使 poll() 即使在信号到达之前也可以立即返回。

2. 使用 epoll() 克服竞态

对于使用 epoll() 的实现，它通过采用不同的技术克服了这个竞态条件：通过保持 SIGURG 信号被阻塞，并从一个 signalfd() 描述符消费信号，而不是注册一个信号处理程序或创建自己的自管道。系统假设任何拥有 Linux epoll() 的系统也会有 Linux signalfd()。

3. Windows 实现

Windows 的实现使用的是 Windows 事件，这些事件由所有的 postmaster 子进程继承。在

Windows 平台上没有必要使用自管道技巧。

2.2、自旋锁(spinLock)

自旋锁在postgres中使用非常多，尤其是在申请资源或对资源进行访问的场景。自旋锁用于防止频繁地进行线程的上下文切换，通过循环检查锁的状态使自身一直保持忙而等待的状态，直到锁被释放。由于其效率高、不需要进行系统调用的优点，广泛地应用于短时间内需要保持资源同步的场景以及短时间锁竞争的情况。

在并发编程中，自旋锁是一种低水平的同步机制，用于保护共享资源。不同于在等待释放锁的过程中使线程进入睡眠状态，自旋锁通过不断循环检查锁的状态（自旋）来避免上下文切换，直到获取到锁。自旋锁适合于那些锁持有时间短且线程不希望在重新调度上花费太多时间的场景。PostgreSQL中，自旋锁用于保护简短的代码段访问非常频繁的资源。

在\src\backend\storage\ipc\sheme.c文件中有系统获分配共享内存，并占据该内存获取中的一段代码。其中使用了自旋锁。

```
1 static void *
2 ShmemAllocRaw(Size size, Size *allocated_size)
3 {
4     Size      newStart;
5     Size      newFree;
6     void      *newSpace;
7
8     /*
9      * Ensure all space is adequately aligned. We used to only MAXALIGN t
10     his
11     * space but experience has proved that on modern systems that is not
12     good
13     * enough. Many parts of the system are very sensitive to critical da
14     ta
15     * structures getting split across cache line boundaries. To avoid th
16     at,
17     * attempt to align the beginning of the allocation to a cache line
18     * boundary. The calling code will still need to be careful about ho
19     w it
20     * uses the allocated space - e.g. by padding each element in an arra
21     y of
22     * structures out to a power-of-two size - but without this, even that
23     * won't be sufficient.
24     */
25     size = CACHELINEALIGN(size);
26     *allocated_size = size;
27
28     Assert(ShmemSegHdr != NULL);
29
30     SpinLockAcquire(ShmemLock);
31
32     newStart = ShmemSegHdr->freeoffset;
33
34     newFree = newStart + size;
35     if (newFree <= ShmemSegHdr->totalsize)
36     {
37         newSpace = (void *) ((char *) ShmemBase + newStart);
38         ShmemSegHdr->freeoffset = newFree;
39     }
40     else
41         newSpace = NULL;
42
43     SpinLockRelease(ShmemLock);
44
45     /* note this assert is okay with newSpace == NULL */
```

```

40     Assert(newSpace == (void *) CACHELINEALIGN(newSpace));
41
42     return newSpace;
43 }

```

当前自旋锁的使用用于保护连续内存申请，以及保证内存申请操作的原子性，防止资源浪费。

2.3、轻量级锁(Lightweight Locks)

LWLocks是PostgreSQL特有的一种锁机制，设计用来在进程间同步对共享内存结构的访问，兼顾了性能与灵活性。与自旋锁不同，当一个进程无法立即获得LWLock时，它会进入睡眠状态，从而释放CPU资源供其他进程使用。这使得LWLocks适用于预期等待时间较长的场景。LWLocks支持共享（S）和排他（X）两种锁模式，使得它们能够适应不同的同步需求。LWLock 采用原子操作和等待队列来确保锁获取和释放的原子性和安全性，同时提供独占锁和共享锁模式，以及等待变量更新的功能。

轻量级锁不需要进行系统调用，不同于普通锁，其占用资源为非阻塞行为，如果锁被占用，则会一直循环检查锁状态，直到锁被释放，同时防止频繁的上下文切换。而不是直接进行睡眠状态，这也就是称其为轻量级的原因。在\src\backend\storage\mgr\lwlock.c中有对轻量级锁具体实现的大部分代码。其主要实现的代码逻辑为：

1. LWLock 使用一个原子变量 state 来维护锁状态，包括独占锁、共享锁、等待队列状态、释放标志等信息。
2. 使用原子操作来获取和释放锁，以确保操作的原子性。
3. 使用一个等待队列来管理等待锁的进程。
4. 为了避免出现进程被唤醒后，锁已经被释放的竞争情况，LWLock 使用了一个两阶段的锁定机制：
 - 第一阶段：尝试原子地获取锁，如果失败则进入第二阶段。
 - 第二阶段：将当前进程添加到等待队列，并再次尝试获取锁，如果成功则从等待队列中移除。
5. 当释放锁时，LWLock 会唤醒等待队列中的进程。

轻量级锁在PostgreSQL中主要用于保护共享内存的数据结构，例如共享缓冲区、目录缓存和事务状态信息等。这里简单给出一个在共享内存开辟时使用轻量级锁的例子。

```

1  /*
2  * ShmemInitStruct -- Create/attach to a structure in shared memory.
3  *
4  *     This is called during initialization to find or allocate
5  *     a data structure in shared memory.  If no other process
6  *     has created the structure, this routine allocates space
7  *     for it.  If it exists already, a pointer to the existing
8  *     structure is returned.
9  *
10 * Returns: pointer to the object.  *foundPtr is set true if the object
    was
11 *     already in the shmem index (hence, already initialized).
12 *
13 * Note: before Postgres 9.0, this function returned NULL for some failure
14 * cases.  Now, it always throws error instead, so callers need not check
15 * for NULL.
16 */
17 void *
18 ShmemInitStruct(const char *name, Size size, bool *foundPtr)
19 {
20     ShmemIndexEnt *result;
21     void *structPtr;
22
23     LWLockAcquire(ShmemIndexLock, LW_EXCLUSIVE);
24
25     if (!ShmemIndex)
26     {
27         PGShmemHeader *shmemseghdr = ShmemSegHdr;
28
29         /* Must be trying to create/attach to ShmemIndex itself */
30         Assert(strcmp(name, "ShmemIndex") == 0);
31
32         if (IsUnderPostmaster)
33         {
34             /* Must be initializing a (non-standalone) backend */
35             Assert(shmemseghdr->index != NULL);
36             structPtr = shmemseghdr->index;
37             *foundPtr = true;
38         }
39         else
40         {
41             /*
42

```

```

43         * If the shmem index doesn't exist, we are bootstrapping: w
44 e must
45         * be trying to init the shmem index itself.
46         *
47         * Notice that the ShmemIndexLock is released before the shme
48 m
49         * index has been initialized. This should be OK because no
50 other
51         * process can be accessing shared memory yet.
52         */
53         Assert(shmemseghdr->index == NULL);
54         structPtr = ShmemAlloc(size);
55         shmemseghdr->index = structPtr;
56         *foundPtr = false;
57     }
58     LWLockRelease(ShmemIndexLock);
59     return structPtr;
60 }
61
62 /* look it up in the shmem index */
63 result = (ShmemIndexEnt *)
64     hash_search(ShmemIndex, name, HASH_ENTER_NULL, foundPtr);
65
66 if (!result)
67 {
68     LWLockRelease(ShmemIndexLock);
69     ereport(ERROR,
70             (errcode(ERRCODE_OUT_OF_MEMORY),
71              errmsg("could not create ShmemIndex entry for data struc
72 ture \"%s\"",
73                  name)));
74 }
75
76 if (*foundPtr)
77 {
78     /*
79      * Structure is in the shmem index so someone else has allocated
80      * already. The size better be the same as the size we are tryin
81      * initialize to, or there is a name conflict (or worse).
82      */
83     if (result->size != size)
84     {
85         LWLockRelease(ShmemIndexLock);
86         ereport(ERROR,
87                 (errmsg("ShmemIndex entry size is wrong for data stru
88 cture"

```

```

84         " \"%s\": expected %zu, actual %zu",
85         name, size, result->size));
86     }
87     structPtr = result->location;
88 }
89 else
90 {
91     Size      allocated_size;
92
93     /* It isn't in the table yet. allocate and initialize it */
94     structPtr = ShmemAllocRaw(size, &allocated_size);
95     if (structPtr == NULL)
96     {
97         /* out of memory; remove the failed ShmemIndex entry */
98         hash_search(ShmemIndex, name, HASH_REMOVE, NULL);
99         LWLockRelease(ShmemIndexLock);
100        ereport(ERROR,
101                (errcode(ERRCODE_OUT_OF_MEMORY),
102                 errmsg("not enough shared memory for data structure"
103                        " \"%s\" (%zu bytes requested)",
104                         name, size)));
105    }
106    result->size = size;
107    result->allocated_size = allocated_size;
108    result->location = structPtr;
109 }
110
111 LWLockRelease(ShmemIndexLock);
112
113 Assert(ShmemAddrIsValid(structPtr));
114
115 Assert(structPtr == (void *) CACHELINEALIGN(structPtr));
116
117 return structPtr;
}

```

ShmemInitStruct函数用于在共享内存中创建或找到一个数据结构。如果该结构已经被其他进程创建，此函数会返回指向该结构的指针；如果该结构还不存在，此函数会在共享内存中为其分配空间。这个函数的关键点之一是它使用轻量级锁（LWLock）来保护共享内存索引ShmemIndex，确保共享内存索引的操作是原子的和互斥的。由于 ShmemIndex 是一个共享内存结构，多个进程可能同时访问它，因此需要使用锁来保护它，避免出现数据冲突。

2.4、锁组

锁组是PostgreSQL中的一个较新概念，它允许将多个锁逻辑上分组为一个单元来管理。这种机制可以用来简化锁的管理和降低锁的粒度，提高系统的并发能力。通过将相关的锁分配给同一个锁组，可以有效管理锁之间的依赖和优先级，以避免死锁和减少锁的争用。

锁组具体机制如下：在 PostgreSQL 中，每个进程都有一个 `lockGroupLeader` 字段，用于记录该进程所属的锁组的Leader。每个进程都有一个 `lockGroupLink` 链接，用于将该进程链接到锁组的成员链表 `lockGroupMembers` 中。当一个进程尝试获取一个锁时，如果该锁已经被同一个锁组的其他成员持有，则该进程会等待直到该锁被释放。这种机制可以有效地防止死锁，因为锁组中的所有进程都会按照相同的顺序获取锁。

在src\backend\storage\lmgr\proc.c文件中有在进程中有关锁组的相关信息。

```
1  /*
2   * BecomeLockGroupLeader - designate process as lock group leader
3   *
4   * Once this function has returned, other processes can join the lock group
5   * by calling BecomeLockGroupMember.
6   */
7  void
8  BecomeLockGroupLeader(void)
9  {
10     LWLock    *leader_lwlock;
11
12     /* If we already did it, we don't need to do it again. */
13     if (MyProc->lockGroupLeader == MyProc)
14         return;
15
16     /* We had better not be a follower. */
17     Assert(MyProc->lockGroupLeader == NULL);
18
19     /* Create single-member group, containing only ourselves. */
20     leader_lwlock = LockHashPartitionLockByProc(MyProc);
21     LWLockAcquire(leader_lwlock, LW_EXCLUSIVE);
22     MyProc->lockGroupLeader = MyProc;
23     dlist_push_head(&MyProc->lockGroupMembers, &MyProc->lockGroupLink);
24     LWLockRelease(leader_lwlock);
25 }
26
27 /*
28  * BecomeLockGroupMember - designate process as lock group member
29  *
30  * This is pretty straightforward except for the possibility that the leader
31  * whose group we're trying to join might exit before we manage to do so;
32  * and the PGPROC might get recycled for an unrelated process. To avoid
33  * that, we require the caller to pass the PID of the intended PGPROC as
34  * an interlock. Returns true if we successfully join the intended lock
35  * group, and false if not.
36  */
37 bool
38 BecomeLockGroupMember(PGPROC *leader, int pid)
39 {
40     LWLock    *leader_lwlock;
41     bool       ok = false;
42
43     /* Group leader can't become member of group */
```

```

44     Assert(MyProc != leader);
45
46     /* Can't already be a member of a group */
47     Assert(MyProc->lockGroupLeader == NULL);
48
49     /* PID must be valid. */
50     Assert(pid != 0);
51
52     /*
53      * Get lock protecting the group fields. Note LockHashPartitionLockBy
54 Proc
55      * calculates the proc number based on the PGPROC slot without lookin
56 g at
57      * its contents, so we will acquire the correct lock even if the leade
58 r
59      * PGPROC is in process of being recycled.
60      */
61     leader_lwlock = LockHashPartitionLockByProc(leader);
62     LWLockAcquire(leader_lwlock, LW_EXCLUSIVE);
63
64     /* Is this the leader we're looking for? */
65     if (leader->pid == pid && leader->lockGroupLeader == leader)
66     {
67         /* OK, join the group */
68         ok = true;
69         MyProc->lockGroupLeader = leader;
70         dlist_push_tail(&leader->lockGroupMembers, &MyProc->lockGroupLink
71 );
72     }
73     LWLockRelease(leader_lwlock);
74
75     return ok;
76 }

```

上面这段函数具体展示了如何将进程标记为锁组的领导者（Leader）和成员（Member）。完成了锁组的具体实现，允许一组进程共享锁资源，通过轻量级锁（LWLock）保障操作的原子性和一致性。减少进程间切换的开销，提高锁访问的效率。

BecomeLockGroupLeader 函数的目标是将当前进程设置为锁组领导者。如果一个进程已经是锁组领导者，该函数直接返回。

- 首先检测当前进程（MyProc）是否已经是其自己锁组的领导者，如果是，则无需再次执行；
- 检查当前进程是否已经是某个锁组的成员。如果是，则断言出错，因为一个进程不能同时是成员和领导者；
- 函数为进程创建了一个只包含自己的单成员锁组，将 MyProc->lockGroupLeader 设置为自己，并

将自己加入到锁组成员的双向链表中；

- 为了确保操作的原子性和一致性，操作前会获取一个轻量级锁（LWLock），操作完毕后释放该锁。

BecomeLockGroupMember 函数的目的是将当前进程添加到一个现有的锁组作为成员。通过指定的锁组领导者 PGPROC 结构和进程ID来验证和加入。

- 函数开始时，检查当前进程是否是领导者或已经是某个锁组的成员，如果是，则出现断言错误；
- 断言验证传入的进程ID有效，即不为0；
- 获取相关的轻量级锁（LWLock）来保护成员加入操作的一致性；
- 验证指定的领导者进程是不是我们想要加入的锁组的领导者，主要通过比较进程ID和锁组领导状态；
- 如果以上条件满足，当前进程加入该锁组，将 `MyProc->lockGroupLeader` 设置为该组领导者，同时将自己加到锁组成员的链表中；
- 操作完成后释放之前获取的轻量级锁。

2.5、同步多进程并发的barrier机制

屏障允许多个进程或线程在执行并行任务时进行同步，确保所有参与者在继续执行下一阶段之前都能达到某个特定的同步点。在Postgres中，这种机制对于实现高效且一致的并行操作至关重要。

这种屏障同步机制适用于需要分阶段执行的并行算法，其中每个阶段的输出是下一阶段开始之前所必需的。例如，一个并行排序算法可能分为多个阶段：划分数据、局部排序、合并结果等，每个阶段都必须等待所有参与进程完成当前阶段的工作才能共同进入下一阶段。

Barrier 结构

在src\include\storage\barrier.h文件中，定义了Postgres中的barrier结构体

```
1  typedef struct Barrier
2  {
3      slock_t      mutex;
4      int          phase;          /* phase counter */
5      int          participants;    /* the number of participants attached */
6      int          arrived;        /* the number of participants that have
7                                   * arrived */
8      int          elected;         /* highest phase elected */
9      bool         static_party;   /* used only for assertions */
10     ConditionVariable condition_variable;
11 } Barrier;
```

- mutex: 一个自旋锁，用于保护屏障内部状态的访问。
- participants: 一个整数，表示屏障中预期参与者的数量。
- arrived: 一个整数，记录当前已经到达屏障的参与者数量。
- phase: 一个整数，表示当前的阶段。
- elected: 一个整数，表示当前阶段被选中的参与者的阶段号。
- static_party: 一个布尔值，表示该屏障是否是静态的。
- condition_variable: 一个条件变量，用于等待其他参与者到达屏障。

核心功能描述

在src\backend\storage\ipc\barrier.c文件中，定义了进程间通过barrier机制进行同步的具体过程。

静态和动态参与者：支持静态和动态的参与者集合。静态参与者集合预先知道参与者数量，而动态参与者允许进程随时加入或离开同步点。同时，使用一个自旋锁 mutex 来保护内部状态的访问，并使用一个条件变量 condition_variable 来等待其他参与者到达屏障。

阶段跟踪：在动态参与者的情况下，通过阶段号对并行算法的进度进行跟踪，以便新加入的进程能够同步到当前的算法阶段。

屏障操作，使用屏障机制的核心功能实现有以下几个步骤：

- **初始化 (BarrierInit)：**初始化屏障，设置参与者数量。
- **到达并等待 (BarrierArriveAndWait)：**参与者到达屏障并等待，直到所有参与者都到达后才能继续执行。此操作会增加当前阶段号。如果当前到达的参与者数量小于预期的参与者数量，则该函数会进入等待状态，直到所有参与者都到达屏障。如果当前到达的参与者数量等于屏障中预期的参与者数量，则该函数会唤醒所有等待的参与者，并将屏障的阶段号加 1。
- **到达并分离 (BarrierArriveAndDetach)：**参与者到达屏障并从屏障中分离。
- **加入 (BarrierAttach)：**允许参与者加入屏障，用于动态参与者的情况。
- **分离 (BarrierDetach)：**允许参与者从屏障中分离，如果是最后一个分离的参与者，可能会触发其他等待的参与者继续执行。

在并行计算中，屏障可以用来同步多个进程或线程，例如：在并行排序算法中，每个进程负责排序一部分数据，使用屏障来同步所有进程完成排序操作。在并行矩阵计算中，每个进程负责计算矩阵的一部分，使用屏障来同步所有进程完成计算操作。

在系统执行任务中，屏障可以用来同步多个后台进程的操作，例如：在 WAL 日志写入中，多个后台进程可能会同时写入 WAL 日志，使用屏障来确保所有进程都写入相同的日志数据。

三、信号(Signal)

2.1、原始信号量的PV操作

原始信号量的 P 和 V 操作是进程同步和互斥的两种基本操作，用于控制对共享资源的访问。它们来自荷兰语 "Proberen"（测试）和 "Verhogen"（增加），在计算机科学中通常表示为 P（等待）操作和 V（释放）操作。这一概念最早由荷兰计算机科学家 Edsger W. Dijkstra 在1960年代提出。

原始信号量 S 是一个具有非负整数值的变量，它有两个原子操作：P（等待）和 V（释放）。

P（Proberen，测试/等待）操作

1. 检查信号量（资源计数）的值。
2. 如果信号量的值大于零（表示有可用资源），则进程让信号量的值减一，并继续执行。
3. 如果信号量的值为零（没有可用资源），则进程进入等待状态，直到信号量的值不为零（某个资源被释放）。

V（Verhogen，增加/释放）操作

1. 将信号量的值增加一。
2. 如果有进程因请求该资源而被阻塞，唤醒等待队列中的一个（或多个）进程。

2.2、现代系统的两种信号量

在计算机系统中，进程间通信（IPC）是一个关键的功能，它允许多个进程共享信息和资源。两种常见的IPC方法是System V IPC 和 POSIX IPC。虽然这两种方法都能够实现进程间通信，但它们之间存在着一些重要的差异。

2.2.1、System V IPC

System V IPC（Inter-Process Communication）是Unix系统中用于进程间通信的一组机制。它们最初是在System V操作系统中引入的，包括消息队列、信号量和共享内存。这些机制在创建时会产生一个非负整数的键，用于标识IPC对象。IPC对象的生命周期超过了原始进程的生命周期，除非显式删除，否则即使所有进程都已关闭，它们也将持续存在。

2.2.2、POSIX IPC

POSIX IPC（Portable Operating System Interface for UNIX Inter-Process Communication）是POSIX标准定义的进程间通信机制。它与System V IPC有一些相似之处，但也有许多不同之处，主要体现在API设计、功能和使用方式上。与System V IPC 不同，POSIX IPC对象的生命周期默认不会超过原始进程的生命周期，除非明确设置为持久模式。

2.3、信号量的演化

原始信号量的P（等待）和V（释放）操作，作为进程间同步和互斥的手段，为IPC的发展奠定了基础。从最初的原始信号量操作演化到现代的IPC机制，经历了多个阶段，涉及多种技术和方法。

信号量与互斥锁

原始信号量的概念被进一步扩展为计数信号量和二进制信号量（或称互斥锁），用于控制对共享资源的同步访问。计数信号量允许多个资源的的同时访问，而二进制信号量则限制了同时访问共享资源的进程数量为1，直观上类似于一种锁。这为复杂的进程同步和互斥问题提供了解决方案。

消息传递

除了信号量和互斥锁，消息传递成为了一种普遍的IPC机制。进程通过发送和接收消息来交换数据，不直接共享内存。这种方式在分布式系统中尤为重要，因为它允许位于不同物理位置的进程通信。

共享内存

与消息传递相反，共享内存方法允许多个进程访问同一块内存区域，从而共享数据。为了保证数据一致性和避免竞态条件，通常需要结合互斥锁或信号量来同步对共享内存的访问。

管道

管道是UNIX和Linux系统中最早的IPC机制之一，允许一个进程的输出直接作为另一个进程的输入。管道可以是匿名的，也可以是有名的（FIFO，先入先出）。尽管简单，但管道在许多情况下仍然非常有效。

2.4、postgres中信号的使用

互斥锁（Mutex Locks）

PostgreSQL 使用互斥锁来保护对共享内存结构的访问，以确保多个后台进程（如数据库查询处理器、自动清理程序等）不会同时修改同一个数据结构。例如，当一个进程需要修改一个共享缓冲区中的数据页时，它必须首先获得该页面的互斥锁。

轻量级锁（Lightweight Locks）

PostgreSQL 扩展了传统的互斥锁概念，引入了轻量级锁（LWLock），用于控制对数据结构的高频率访问。轻量级锁是一种更精细的锁，设计用来提高并发访问时的性能，并减少等待时间。

信号量（Semaphores）

虽然 PostgreSQL 内部使用的信号量可能不直接暴露给用户，但它们在数据库的并发控制中起着关键作用。例如，它们用于控制后台进程的同步，比如在写入前确保缓冲区有足够的空间可用。

消息传递

PostgreSQL 的进程间通信（IPC）机制广泛地使用了消息传递。例如，后台工作进程和守护进程之间会通过消息队列进行通信，以进行任务调度、状态报告和错误通知等。

共享内存

PostgreSQL 使用共享内存来使得运行在不同进程中的数据库组件能够高效地交换信息。共享内存被用来存储控制信息（比如轻量级锁的状态）、缓存共享数据（如数据库缓存）等，以提供高速的数据访问和减少数据复制。

2.5、常见信号量

信号量名称	PG中的作用	信号量介绍
SIGUSR1	后端进程（Backend）与 Postmaster进程之间的通信信号	在内核中，SIGUSR1和SIGUSR2都是用户自定义信号，它们最初被设计为允许程序员自定义信号的用途，以实现进程间通信或其他特殊操作。
SIGQUIT	发送SIGQUIT信号给 Postmaster进程。Postmaster进程会将SIGQUIT信号传递给所有正在运行的后端进程，强制它们立即停止运行，并关闭与数据库的连接。	SIGQUIT信号最初被设计为一种比 SIGINT（中断信号）更强制的退出机制。
PROCSIG_CATCHUP_INTERRUPT	用于通知进程进行数据同步。	当一个后端进程需要追赶到主服务器的状态时，例如在流复制或热备份期间，它会收到这个信号。
PROCSIG_NOTIFY_INTERRUPT	用于通知进程进行异步通知。	当有异步通知事件(例如，通过 LISTEN/NOTIFY命令)发生时，相应的进程会接收到此信号。
PROCSIG_PARALLEL_MESSAGE	用于在并行查询中传递信息。	在执行并行查询操作时，worker之间、或者worker与coordinator之间需要传递消息，此信号用于该消息传递。
PROCSIG_WALSND_INIT_STOPPING	用于通知WAL发送器进程停止初始化。	在WAL复制过程中，若需要停止WAL发送器进程的初始化过程，会发送该信号。

PROCSIG_BARRIER	用于广播系统关停原因。	用于执行广播通知，以确保所有进程都可以按顺序、正确地响应系统关停等全局事件。
PROCSIG_LOG_MEMO RY_CONTEXT	用于日志记录内存上下文信息。	提供一种机制，通过该信号允许进程收集和记录其内存使用情况的信息，通常用于调试内存泄露或高内存使用等问题。
PROCSIG_PARALLEL_ APPLY_MESSAGE	用于并行复制中传递信息。	在逻辑复制或并行恢复场景中，worker之间需要传递复制相关的信息时使用。
PROCSIG_RECOVERY_ CONFLICT_DATABASE	用于恢复操作冲突管理。	用于处理数据库级别的恢复冲突，例如当数据库正在尝试从备份中恢复时。
PROCSIG_RECOVERY_ CONFLICT_TABLESPACE	用于表空间恢复操作冲突管理。	当在恢复过程中遇到与表空间相关的冲突，例如尝试访问被删除的表空间时，使用此信号。
PROCSIG_RECOVERY_ CONFLICT_LOCK	用于锁冲突恢复操作管理。	在恢复过程中，如果数据库尝试获取已被其他事务持有的锁，可能会产生锁冲突，此时会发送此信号。
PROCSIG_RECOVERY_ CONFLICT_SNAPSHOT	用于快照恢复操作冲突管理。	当在恢复过程中遇到与快照相关的冲突，比如访问到了不再有效的快照时，使用此信号。
PROCSIG_RECOVERY_ CONFLICT_LOGICALSL OT	用于逻辑复制槽恢复操作冲突管理。	在恢复过程中遇到与逻辑复制槽相关的冲突时（如尝试重用已存在的槽），会发送此信号。
PROCSIG_RECOVERY_ CONFLICT_STARTUP_ DEADLOCK	用于启动时死锁恢复操作管理。	当在系统启动期间检测到可能的死锁，以防止恢复过程中的死锁情形，使用此信号。
PROCSIG_RECOVERY_ CONFLICT_BUFFERPIN	用于缓冲区固定恢复操作冲突管理。	在恢复过程中，如果尝试访问由于先前操作而被固定在内存中的数据，可能会产生缓冲冲突，此时会发送此信号。

2.6、pmsignal的实现

在\src\backend\storage\ipc\pmsignal.c中。pmsignal 是 PostgreSQL 中负责处理 postmaster (PostgreSQL 的主进程) 与其子进程间信号传递的组件。这种机制支持多种不同的通知理由，为 postmaster 主进程与多个子进程之间的进程管理提供了灵活的通信方式。

2.6.1、Postmaster接收和发送信号概述

Pmsignal允许子进程通过设置共享内存中的标志位来向 postmaster 发送不同类型的信号，并通过发送 SIGUSR1 通知 postmaster 检查这些标志位。子进程使用 SIGUSR1 信号通知 Postmaster 某个事件，而 Postmaster 使用 SIGQUIT 信号通知子进程关闭。为了区分不同的事件类型，代码使用共享内存中的 PMSignalData 结构体来存储信号标志。

该结构体包含了 PMSignalFlags 数组、sigquit_reason 字段以及 PMChildFlags 数组。PMSignalFlags 数组用于存储子进程向 Postmaster 发送的信号类型，每个元素对应一个信号类型。sigquit_reason 字段用于存储 Postmaster 向子进程发送 SIGQUIT 信号的原因。PMChildFlags 数组用于存储每个子进程的状态，例如，是否正在使用共享内存、是否为 WAL 发送器等等。

2.6.2、PMSignalData 结构体

```
1  struct PMSignalData
2  {
3      /* per-reason flags for signaling the postmaster */
4      sig_atomic_t PMSignalFlags[NUM_PMSIGNALS];
5      /* global flags for signals from postmaster to children */
6      QuitSignalReason sigquit_reason; /* why SIGQUIT was sent */
7      /* per-child-process flags */
8      int num_child_flags; /* # of entries in PMChildFlags[] */
9      sig_atomic_t PMChildFlags[FLEXIBLE_ARRAY_MEMBER];
10 };
```

1. sig_atomic_t PMSignalFlags[NUM_PMSIGNALS];

这是一个数组，用于存储为每一个可能的信号原因设置的标志。每个标志位用于表示是否有特定原因的信号被发送到 postmaster。sig_atomic_t 保证了这些标志位的读写操作是原子的，避免了并发访问产生的问题。NUM_PMSIGNALS 定义了数组的大小，即支持的信号原因的总数。

2. QuitSignalReason sigquit_reason;

这个字段用于存储发送 SIGQUIT 信号的原因。SIGQUIT 通常用于告知所有子进程进行紧急停止。QuitSignalReason 是一个枚举，定义了可能的停止原因，如发生错误、管理员请求停止服务等。

3. int num_child_flags;

用于记录 PMChildFlags 数组中的元素数量，即当前系统管理下的子进程数量。这个计数有助于遍历子进程状态标志位时的索引控制。

4. sig_atomic_t PMChildFlags[FLEXIBLE_ARRAY_MEMBER];

这是一个灵活数组成员，其大小在运行时确定，用于跟踪每个子进程的状态。这些标志位反映了子进程的不同状态，如是否已分配、是否处于活跃状态等。使用 sig_atomic_t 类型确保了标志位的操作是原子的。

2.6.3、主要功能

- 信号的发送与接收：**子进程通过调用 SendPostmasterSignal 函数，并指定特定的理由码 (PMSignalReason)，来向 postmaster 发送信号。这一动作涉及到在共享内存中设置对应理由的标志位，并向 postmaster 发送 SIGUSR1。
- 信号标志与状态管理：**在共享内存中，使用 volatile sig_atomic_t 类型的 PMSignalFlags 数组存储每个可能的信号理由的标志状态，以确保操作的原子性和线程（或进程）安全。此外，还用 PMChildFlags 数组跟踪每个子进程的状态，比如是否被分配、是否处于活动状态等。
- 子进程状态管理：**pmsignal 提供了一系列函数来管理子进程状态，例如，AssignPostmasterChildSlot 用于分配子进程标志位，MarkPostmasterChildActive 和 MarkPostmasterChildInactive 用于标记子进程的活跃状态等。
- 信号处理程序：**这个模块中还定义了信号处理函数，如 postmaster_death_handler，用于在特定信号到来时更新状态或执行某些特殊操作。
- 系统关机信号：**除了管理子进程之间的通信外，还有机制用于广播系统关机的原因，允许在接收到 SIGQUIT 信号时，子进程了解系统关机的具体原因。

2.6.4、gdb调试跟踪

使用gdb调试postgres

查看与postgres相关的进程，在gdb中进行绑定

```
[postgres@3a6fcfb8b59e ~]$ ps aux | grep postgres
root      5060  0.0  0.0 104800  5640 pts/2    S    05:22   0:00 su - postgres
postgres  5061  0.0  0.0  15104   3760 pts/2    S    05:22   0:00 -bash
postgres  5583  0.1  1.5 454844 125604 pts/2    Sl+  05:23   0:01 gdb ./postgres
postgres  7498  0.0  0.2 219232 20756 pts/2    t    05:25   0:00 /data/postgresbase/debug/bin/postgres
postgres  7502  0.0  0.1 219364  8128 ?        Ss   05:25   0:00 postgres: checkpointer
postgres  7503  0.0  0.0 219388  6132 ?        Ss   05:25   0:00 postgres: background writer
postgres  7504  0.0  0.0  0 0 ?          Zs   05:25   0:00 [postgres] <defunct>
root      16418 0.0  0.0 104800  5792 pts/3    S    05:37   0:00 su - postgres
postgres  16419 0.0  0.0  14984   3648 pts/3    S    05:37   0:00 -bash
postgres  16617 0.0  0.0  47604   3796 pts/3    R+   05:37   0:00 ps aux
postgres  16618 0.0  0.0  12136   1148 pts/3    S+   05:37   0:00 grep --color=auto postgres
[postgres@3a6fcfb8b59e ~]$
```

使用gdb调试postgres，并绑定到postmaster上

```

postgres@3a6fcfb8b59e bin]$ gdb ./postgres
GNU gdb (GDB) Red Hat Enterprise Linux 8.2-16.el8
Copyright (C) 2018 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./postgres...done.
(gdb) break SendPostmasterSignal
Breakpoint 1 at 0x7db2f0: file pmsignal.c, line 184.
(gdb) break CheckPostmasterSignal
Breakpoint 2 at 0x7db320: file pmsignal.c, line 201.
(gdb) c
The program is not being run.
(gdb) run
Starting program: /data/postgresbase/debug/bin/postgres
Missing separate debuginfos, use: yum debuginfo-install glibc-2.28-164.el8.x86_64
[Thread debugging using libthread_db enabled]

```

这里展示了一个后端进程向主进程申请处理的基本过程

```

.3-3.el8_4.x86_64 pcre2-10.32-2.el8.x86_64 systemd-libs-239-51.el8_5.2.x86_64 z
(gdb) b pmsignal.c:SendPostmasterSignal
Breakpoint 1 at 0x7dalc0: file pmsignal.c, line 184.
(gdb) c
Continuing.

Program received signal SIGUSR1, User defined signal 1.
0x00007fadba16a0bb in epoll_wait () from /lib64/libc.so.6
(gdb) n
Single stepping until exit from function epoll_wait,
which has no line number information.
handle_pm_pmsignal_signal (postgres_signal_arg=10) at postmaster.c:2693
2693             int                save_errno = errno;
(gdb) n
2696             SetLatch(MyLatch);
(gdb) n
2698             errno = save_errno;
(gdb) n
0x00007fadba16a0bb in epoll_wait () from /lib64/libc.so.6
(gdb) n
Single stepping until exit from function epoll_wait,
which has no line number information.

Program received signal SIGUSR1, User defined signal 1.
0x00007fadba16a0bb in epoll_wait () from /lib64/libc.so.6

```

1. **接收到 SIGUSR1 信号：**程序在 `epoll_wait` 调用中被 SIGUSR1 信号打断，该信号是用户定义的信号之一，经常用于程序内部的通信目的。
2. **信号处理函数：**`handle_pm_pmsignal_signal` 是一个信号处理函数，这里处理了 SIGUSR1 信号。该函数首先保存了当前的 `errno` 值（为了保持跨系统调用的 `errno` 不变性），然后设置了一个门锁（Latch），最后恢复了 `errno` 的值。
3. **使用 Latch 等待事件：**Latch 是 PostgreSQL 中用于进程间或线程间等待和通知的一种机制。程序在处理完信号后，回到等待事件的地方。`WaitEventSetWaitBlock` 和 `WaitEventSetWait` 这两个函数涉及等待事件集中的事件发生。
4. **程序流程继续：**通过多次接收到 SIGUSR1 信号和处理，程序最后回到了主服务器循环，继续正常的运行。`ServerLoop` 函数负责监听并处理各种事件，包括连接请求、子进程状态变化等。

继续调试，程序在断点处停下，打印标志位信息

```
(gdb) c
Continuing.
[Detaching after fork from child process 20233]
[Detaching after fork from child process 20234]
[Detaching after fork from child process 20235]
2024-06-20 05:42:09.888 UTC [7498] LOG: database system is ready to accept connections

Breakpoint 2, CheckPostmasterSignal (reason=reason@entry=PMSIGNAL_RECOVERY_STARTED) at pmsignal.c:201
201         if (PMSignalState->PMSignalFlags[reason])
(gdb) print (PMSignalState->PMSignalFlags[reason])
$1 = 1
(gdb) □
```

1. 输出结果 `$1 = 1` 证实了这个标志位已经被设置为 `true`，说明 `postmaster` 确实接收到了子进程发送的信号。
2. `PMSignalFlags[reason]` 为1：信号标志被设置，例如在 `PMSIGNAL_RECOVERY_STARTED` 情况下。这可能表示数据库正在从备份中恢复。

继续执行，之后函数循环遍历所有进程的 `PMSignalFlags[reason]` 的信号量，保持 `epoll_wait` 等待直到接收到 SIGUSR1 信号为止。

```
Breakpoint 2, CheckPostmasterSignal (reason=reason@entry=PMSIGNAL_RECOVERY_STARTED) at pmsignal.c:201
201         if (PMSignalState->PMSignalFlags[reason])
(gdb) print (PMSignalState->PMSignalFlags[reason])
$1 = 1
(gdb) continue
Continuing.

Breakpoint 2, CheckPostmasterSignal (reason=reason@entry=PMSIGNAL_BEGIN_HOT_STANDBY) at pmsignal.c:201
201         if (PMSignalState->PMSignalFlags[reason])
(gdb) print (PMSignalState->PMSignalFlags[reason])
$2 = 0
(gdb) continue
Continuing.

Breakpoint 2, CheckPostmasterSignal (reason=reason@entry=PMSIGNAL_BACKGROUND_WORKER_CHANGE) at pmsignal.c:201
201         if (PMSignalState->PMSignalFlags[reason])
```

```

(gdb) print (PMSignalState->PMSignalFlags[reason])
$6 = 0
(gdb) continue
Continuing.

Breakpoint 2, CheckPostmasterSignal (reason=reason@entry=PMSIGNAL_ADVANCE_STATE_MACHINE) at pmsignal.c:201
201         if (PMSignalState->PMSignalFlags[reason])
(gdb) print (PMSignalState->PMSignalFlags[reason])
$7 = 0
(gdb) continue
Continuing.
[Detaching after fork from child process 32516]

Program received signal SIGUSR1, User defined signal 1.
0x00007ffff66c00bb in epoll_wait () from /lib64/libc.so.6
(gdb)
Continuing.

Breakpoint 2, CheckPostmasterSignal (reason=reason@entry=PMSIGNAL_RECOVERY_STARTED) at pmsignal.c:201
201         if (PMSignalState->PMSignalFlags[reason])
(gdb) print (PMSignalState->PMSignalFlags[reason])
$8 = 0

```

之后再对PMSIGNAL_RECOVERY_STARTED的信号量符号位进行打印，打印结果为0。说明系统已经完成执行恢复。

2.7procsignal实现

在\src\backend\storage\ipc\prosignal.c中。prosignal 是 PostgreSQL 中负责管理进程间的信号通信，特别是 postmaster（主进程）和它的子进程之间的信号通信。

2.7.1、procsignal接收和发送信号概述

信号标志（Signal Flags）： 这些标志用来表示特定的事件或操作需要被进程处理。标志存储在共享内存结构 ProcSignalHeader 中的 psh_slot 数组中，每个进程都有一个对应的 ProcSignalSlot。

共享内存（Shared Memory）： 共享内存用于在进程间传递信号和数据。ProcSignal 是一个指向共享内存中 ProcSignalHeader 结构的指针，其中包含所有进程信号槽。

2.7.2、信号处理函数

在src\backend\storage\lmgr\proc.c中

▼ ProcWaitForSignal

```
1 void ProcWaitForSignal(uint32 wait_event_info)
2 {
3     (void) WaitLatch(MyLatch, WL_LATCH_SET | WL_EXIT_ON_PM_DEATH, 0, wait_e
vent_info);
4     ResetLatch(MyLatch);
5     CHECK_FOR_INTERRUPTS();
6 }
```

ProcWaitForSignal 函数用于使一个进程等待其他进程的信号。它依赖于“latch”机制，即一个轻量级的同步机制，用来在进程间传递信号。

- WaitLatch 函数是等待的核心，它使调用进程处于等待状态直到以下任一条件满足：latch 被设置、进程被要求退出、或等待超时（在这个例子中，超时被设置为 0，表示无限等待，直到一个条件被触发）。
- wait_event_info 参数用于诊断目的，表明等待事件的种类。
- ResetLatch 用于清除 latch 状态，准备下一次使用。
- CHECK_FOR_INTERRUPTS 检查是否有中断（如取消或终止信号）需要被处理。

▼ ProcSendSignal

```
1 void ProcSendSignal(ProcNumber procNumber)
2 {
3     if (procNumber < 0 || procNumber >= ProcGlobal->allProcCount)
4         elog(ERROR, "procNumber out of range");
5
6     SetLatch(&ProcGlobal->allProcs[procNumber].procLatch);
7 }
```

ProcSendSignal 通过设置目标进程的 latch 来向其发送信号，从而唤醒等待该 latch 的进程。

- 函数首先检查 procNumber 参数是否有效，以避免数组越界访问。
- SetLatch 函数是实际设置 latch 的操作，使目标进程（如果正在等待信号）能够继续执行。

2.7.3、主要功能

1. **子进程状态管理**：提供一系列函数来管理子进程状态，例如：AssignPostmasterChildSlot：分配一个新的子进程标志位。ReleasePostmasterChildSlot：释放子进程标志位。
MarkPostmasterChildActive：标记子进程处于活动状态。MarkPostmasterChildInactive：标记子进程不再活动。IsPostmasterChildWalSender：检查子进程是否为 WAL 发送器进程。
2. **系统关机信号**：使用 PROCSIG_BARRIER 信号类型来广播系统关机。EmitProcSignalBarrier 函数

用于向所有进程发送 PROCSIG_BARRIER 信号，并等待所有进程都接收并处理了信号。

WaitForProcSignalBarrier 函数用于等待 PROCSIG_BARRIER 信号的传播，确保所有进程都收到了信号。

3. **信号处理**：利用procsignal_sigusr1_handler 函数，来处理 不同的SIGUSR1 信号，根据不同的信号理由码执行相应的操作。

2.7.4、gdb调试跟踪

调试跟踪AssignPostmasterChildSlot函数分配新的子进程标志位的过程如下

```
(gdb) c
Continuing.
[Detaching after fork from child process 55105]
[Detaching after fork from child process 55106]

Breakpoint 1, AssignPostmasterChildSlot () at pmsignal.c:249
249             int                slot = next_child_inuse;
(gdb) n
258             for (n = num_child_inuse; n > 0; n--)
(gdb) n
260                 if (--slot < 0)
(gdb) n
262                 if (!PMChildInUse[slot])
(gdb) n
264                     PMChildInUse[slot] = true;
(gdb) print slot
$4 = 241
(gdb) print n
$5 = 244
(gdb) print PMChildInUse[slot]
$6 = false
```

通过检查该槽位（slot）在数组PMChildInUse中是否已被标记为使用（即值为true）。当发现槽位未被使用（!PMChildInUse[slot]为true），则将该槽位标记为使用（PMChildInUse[slot] = true）。查看变量slot的当前值，确认分配给子进程的槽位号码。从而完成给子进程分配slot标记。

四、消息队列

3.1、消息

在 PostgreSQL中，每一个进程都有属于自己的共享缓存。例如，同一个系统表在不同的进程中都有对应的Cache来缓存它的元组（对于RelCache来说缓存的是一个RelationData结构）。同一个系统表的元组可能同时被多个进程的Cache所缓存，当其中某个Cache中的一个元组被删除或更新时，需要通知其他进程对其Cache进行同步。在 PostgreSQL的实现中，会记录下已被删除的无效元组，并通过SI Message方式(即共享消息队列方式)在进程之间传递这一消息。收到无效消息的进程将同步地把无效元组(或RelationData结构)从自己的Cache中删除。

3.2、单读单写共享消息队列

这种单读单写共享内存消息队列主要被用在PostgreSQL的逻辑复制中，使得用户后端进程能够与它注册的工作进程安全高效地通信。它允许细粒度的控制消息的发送和接收，具备高性能的特点，特别适用于需要高速数据交换的场景。

在src\backend\storage\ipc\shm_mq.c中，可以看到利用环形缓冲区、自旋锁、原子操作和进程间信号机制来创建的单读单写共享消息队列机制相关函数，以确保数据一致性、线程安全和高效通信。

3.2.1、shm_mq 结构:

该结构体用于表示共享内存消息队列，存储在共享内存中。

成员变量:

- mq_mutex: 一个自旋锁，用于保护队列头部的访问。
- mq_receiver: 一个 PGPROC 指针，指向接收消息的进程。
- mq_sender: 一个 PGPROC 指针，指向发送消息的进程。
- mq_bytes_read: 一个 64 位原子变量，记录已经读取的消息字节数。
- mq_bytes_written: 一个 64 位原子变量，记录已经写入的消息字节数。
- mq_ring_size: 队列的环形缓冲区大小。
- mq_detached: 一个布尔值，表示队列是否已经分离。
- mq_ring_offset: 环形缓冲区的起始偏移量。
- mq_ring: 环形缓冲区，用于存储消息数据。

```
1  struct shm_mq
2  {
3      slock_t      mq_mutex;
4      PGPROC      *mq_receiver;
5      PGPROC      *mq_sender;
6      pg_atomic_uint64 mq_bytes_read;
7      pg_atomic_uint64 mq_bytes_written;
8      Size        mq_ring_size;
9      bool        mq_detached;
10     uint8        mq_ring_offset;
11     char         mq_ring[FLEXIBLE_ARRAY_MEMBER];
12 };
```

3.2.2、shm_mq_handle 结构:

该结构体用于表示后台进程对共享内存消息队列的访问句柄。

成员变量：

- mqh_queue: 指向共享内存消息队列的指针。
- mqh_segment: 指向包含共享内存消息队列的动态共享内存段的指针。
- mqh_handle: 指向与该消息队列相关联的后台进程句柄的指针。
- mqh_buffer: 一个指向后台进程本地缓冲区的指针，用于存储接收到的消息。
- mqh_buflen: 后台进程本地缓冲区的大小。
- mqh_consume_pending: 记录已经从队列读取但尚未更新到 mq_bytes_read 的字节数。
- mqh_send_pending: 记录已经写入队列但尚未更新到 mq_bytes_written 的字节数。
- mqh_partial_bytes: 记录当前正在读取或写入的消息字节数。
- mqh_expected_bytes: 记录预期要读取的消息总字节数。
- mqh_length_word_complete: 一个布尔值，表示是否已经读取或写入整个消息长度信息。
- mqh_counterparty_attached: 一个布尔值，表示对方进程是否已经连接到队列。
- mqh_context: 记录创建该句柄时的内存上下文。

```
1  struct shm_mq_handle
2  {
3      shm_mq      *mqh_queue;
4      dsm_segment *mqh_segment;
5      BackgroundWorkerHandle *mqh_handle;
6      char        *mqh_buffer;
7      Size        mqh_buflen;
8      Size        mqh_consume_pending;
9      Size        mqh_send_pending;
10     Size        mqh_partial_bytes;
11     Size        mqh_expected_bytes;
12     bool        mqh_length_word_complete;
13     bool        mqh_counterparty_attached;
14     MemoryContext mqh_context;
15 };
```

3.2.2、主要功能

发送和接收消息：共享内存消息队列使用环形缓冲区来存储消息数据。发送进程将消息写入环形缓冲区，并更新 mq_bytes_written 指针。接收进程从环形缓冲区读取消息，并更新 mq_bytes_read 指针。使用自旋锁 mq_mutex 和原子操作来同步发送进程和接收进程的操作，以确保数据一致性。

进程同步：如果环形缓冲区满了，发送进程会等待接收进程读取数据；如果环形缓冲区空了，接收进程会等待发送进程写入数据。使用 SetLatch 和 WaitLatch 来实现进程间的信号机制，以便在必要时唤醒等待的进程。

这种单读单写共享内存消息队列主要被用在PostgreSQL的逻辑复制中，使得用户后端进程能够与它注册的工作进程安全高效地通信。它允许细粒度的控制消息的发送和接收，具备高性能的特点，特别适用于需要高速数据交换的场景。

五、无效消息(SI Message)

共享缓存失效（Shared Cache Invalidation）是Postgres中一个重要的机制，它确保当数据库中的数据被修改后，所有的数据库后台进程（backend processes）能够获悉这一变化，并且能够采取相应的措施来保持数据的一致性。SI 消息的发送、接收和处理过程保证了不同后台进程之间共享缓存的一致性，提高了数据库系统的性能和可靠性。

SI 消息用于实现各种数据库对象（例如表、索引、视图等）的缓存失效机制，例如：当一个后台进程修改了表中的数据时，它会发送一个 SI 消息，通知其他后台进程更新其缓存中的表数据。当一个后台进程创建了一个新的索引时，它会发送一个 SI 消息，通知其他后台进程更新其缓存中的索引信息。

SI Message

在src\backend\storage\ipc\sinval.c中写了共享缓存失效全局代码，包括发送和接收消息、中断处理以及全局变量的定义

1. **发送共享失效消息 (SendSharedInvalidMessages)：**当数据库中的数据发生变更，需要通知其他后台进程时，该函数被调用来添加一条或多条失效消息到全局共享失效消息队列中。
2. **接收共享失效消息 (ReceiveSharedInvalidMessages)：**后台进程调用该函数来处理等待处理的共享失效消息。这包括调用回调函数来处理具体的失效消息以及处理全局缓存重置的情况。
3. **追赶中断处理 (HandleCatchupInterrupt, ProcessCatchupInterrupt)：**为了避免在后台进程闲置时错过失效事件，系统会通过发送PROCSIG_CATCHUP_INTERRUPT信号给那些落后太多的后台进程，强制它们更新到最新的状态。相关函数负责处理这种“追赶”中断，确保后台进程能够及时地接收并处理消息。
4. **全局变量：**如SharedInvalidMessageCounter用于追踪共享失效消息的数量；catchupInterruptPending标志表示是否有待处理的追赶中断。

SI Message定义函数

在src\backend\storage\ipc\signalfuncs.c中写了几个相关函数，许用户使用 SQL 命令来控制其他后台进程

1. **发送信号给后端进程 (pg_signal_backend)**: 这个函数是核心实现，它根据传入的进程ID (PID) 和信号类型，完成向特定后端进程发送指定信号的功能。它还包含了权限检查逻辑，确保只有拥有足够权限的用户才能发送信号。例如，只有超级用户才能向拥有超级用户权限的后端进程发送信号。
2. **取消后端查询 (pg_cancel_backend)**: 提供了一个SQL可调用的接口，允许通过传入后端进程ID来取消该进程正在执行的查询。这通常用于长时间运行的查询或误操作。
3. **终止后端进程 (pg_terminate_backend)**: 类似于pg_cancel_backend，但这个函数会要求后端进程终止运行。它还支持一个超时参数，允许等待一定时间确认进程是否成功终止。
4. **等待进程终止 (pg_wait_until_termination)**: 这个辅助函数实现了在给定超时时间内等待特定PID的进程终止。如果进程在超时时间内未终止，则发出警告。
5. **重新加载数据库配置 (pg_reload_conf)**: 向postmaster进程发送SIGHUP信号，请求重新加载数据库配置文件（如postgresql.conf）。这对于动态调整配置参数非常有用。
6. **Rotate log file (pg_rotate_logfile)**: 请求日志收集器进程轮换日志文件，这在管理由于数据库活动导致持续增长的日志文件时十分有用。

代码提供了直观且强大的机制来控制和管理PostgreSQL的后端进程，包括权限检查来确保操作的安全性。这些功能对于数据库管理员在管理和维护数据库时构成了基础工具，帮助他们执行一系列的维护任务。

SI Message核心实现

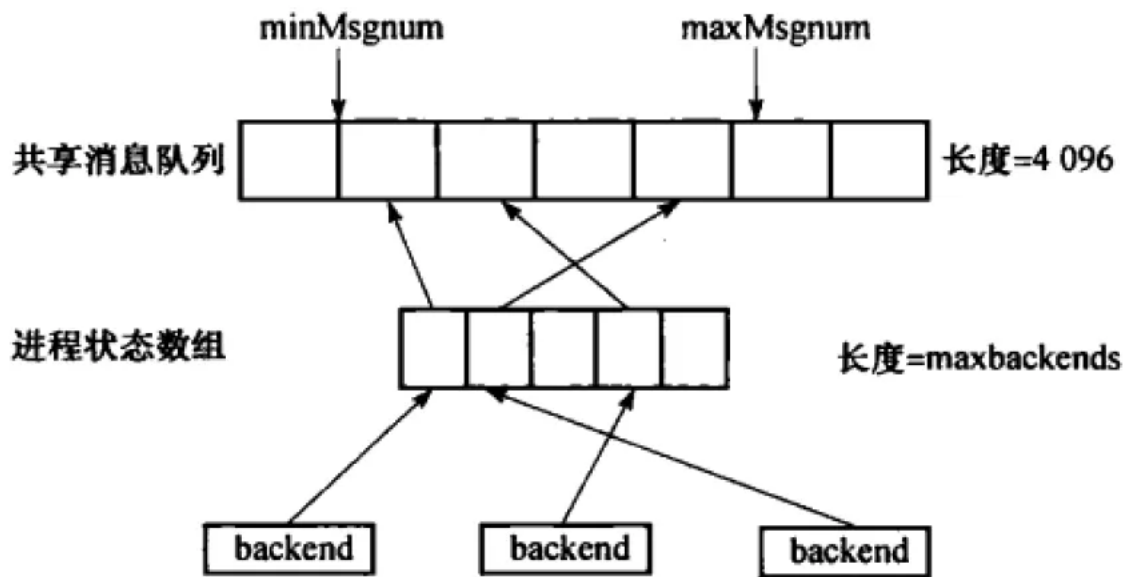
在src\backend\storage\ipc\sinvaladt.c中定义了管理si Message的核心功能实现，包括SI 消息的存储、发送、接收和清理。允许系统通知所有后端进程有关缓存数据变化的信息，确保所有进程的视图都是最新的。

1. **添加新消息 (SIInsertDataEntries)**: 当数据库发生可能影响共享缓存的事件时，新的缓存失效消息将被添加到循环缓冲区中。这个过程需要精心设计，以避免在高并发情况下的性能瓶颈。
2. **获取消息 (SIGetDataEntries)**: 每个后端进程定期检查并处理为其准备的缓存失效消息。处理这些消息是保证数据读取一瞥性的关键步骤。
3. **清理消息队列 (SICleanupQueue)**: 为了避免循环缓冲区溢出，并确保系统的有效运行，当消息被所有相关后端进程处理后，它们需要被清理。同时，为了防止某个进程因长时间未处理导致缓冲区空间不足，系统可能会强制重置其状态或发送“追赶”中断。
4. **生成局部事务ID (GetNextLocalTransactionId)**: 虽然与缓存失效消息处理不直接相关，但局部事务ID的生成是事务管理中的一个重要方面，这也展示了系统如何平衡在不需要全局锁定情况下生成唯一标识符的能力。

SI 消息用于在不同后台进程之间同步共享缓存的状态。当一个后台进程修改了共享缓存中的数据时，它会向全局 SI 消息队列发送一个 SI 消息，通知其他后台进程更新其缓存。每个后台进程都会定期检查 SI 消息队列，并处理收到的消息。SI 消息缓冲区使用一个环形缓冲区来存储消息，并使用两个锁 (SInvalReadLock 和 SInvalWriteLock) 来保护消息缓冲区的访问。

SICleanupQueue 函数负责清理消息队列，它会移除已经被所有后台进程读取的消息，并根据需要发送 catchup 信号给落后的后台进程，以避免消息队列溢出。为了避免 SI 消息序号溢出，PostgreSQL 会在必要时对所有 SI 消息序号进行减法操作。为了避免一个后台进程长时间处于空闲状态，并导致 SI 消息队列溢出，PostgreSQL 会发送 catchup 信号给落后的后台进程，提醒它们处理 SI 消息。

在详细的处理流程中，后台进程会定期检查并处理共享失效消息队列中的消息。如果某个进程因为处于闲置状态而错过了消息，系统就会通过发送中断信号的方式，迫使其处理积压的失效消息。这个机制确保了数据库系统在高并发情况下仍然能够保持数据的一致性和可靠性。



minMsgnum和MaxMsgnum就像两个指针，它们区分出了哪些无效消息已经被所有的进程读取以及哪些消息还在等待某些进程读取。在 minMsgnum之前的消息已被所有进程读完；maxMsgnum之后的区域尚未使用。两者之间的消息是还没有被所有进程读完的。当有进程调用函数 SendSharedInvalidMessage 将其产生的无效消息添加到 shmInvalBuffer 中时，maxMsgnum 就开始向后移动。SendSharedInvalidMessage 中将调用 SIInsertDataEntries 来完成无效消息的插入。

由于 SI 消息缓冲区是一个环形缓冲区，当 maxMsgNum 超过缓冲区末尾时，它会从缓冲区开头继续写入。为了防止消息队列溢出，SICleanupQueue 函数会定期清理队列，并发送 catchup 信号给落后的后台进程。每个后台进程都有一个 nextMsgNum 指针，指向它需要读取的下一个 SI 消息。

六、并行查询和计算

通过使用共享消息队列实现postgres的并行查询，通过使用barrier计算来实现同步多线程并行计算的功能。

并行查询

PostgreSQL的并行查询通过将一个大的查询操作分割成多个小部分，这些小部分可以并行执行。例如，在执行一个大的表扫描或聚合操作时，查询计划器可能决定将工作分配给多个工作进程，每个进程处理表的一部分。这样，可以同时扫描整个表，显著降低查询时间。

实现查询并行性的关键在于共享消息队列和动态共享内存。

- 共享消息队列允许多个并行工作进程间的通信。例如，一个进程可以通过共享队列向另一个进程发送消息，表明它已完成一部分工作或需要数据。
- 动态共享内存用于在并行工作进程间共享数据。这使得每个进程无需复制整个数据集，而是可以直接访问和处理共享内存中的数据部分。

共享内存消息队列shm_mq，用于通过共享内存传递数据和状态。通过核心函数 shm_mq_receive，可以看到无论是数据还是错误消息都通过该机制来同步；

并行计算同步

在多线程并行计算中，各个线程需要在某些点上同步，以确保数据的一致性和正确的执行顺序。这是通过barrier同步实现的，这是一种同步原语，用于确保所有线程都达到某个执行点后才能继续进行。在PostgreSQL中，这通常用于确保在继续下一步处理之前，所有的工作进程都已完成当前的任务。例如，在并行聚合操作中，每个工作进程可能负责聚合表的一部分，只有当所有进程都完成聚合后，结果才能被合并并返回给客户端。并行查询和计算的实现显著提升了PostgreSQL处理大规模数据集的能力。通过有效利用现代多核CPU的并行处理能力，PostgreSQL能够提供更快的数据分析和处理速度，满足企业级应用的需求。

测试案例

在src\test\modules\test_shm_mq目录下，有对共享消息队列的测试案例。展示了如何使用动态共享内存和共享内存消息队列工具的示例，协调有一个或多个后端进程的用户后台。

执行test_shm_mq检查基本的共享内存消息队列测试。它的参数可能包括消息的大小、要发送的消息内容、等待时间和其他配置选项。这两个都能运行成功。

```

postgres=# SELECT test_shm_mq(1024, '', 2000, 1);
 test_shm_mq
-----
(1 row)

postgres=# SELECT test_shm_mq(1024, 'a', 2001, 1);
 test_shm_mq
-----
(1 row)

```

测试在共享内存队列中发送一个固定大小（1024字节）的消息。通过传递一个空字符串作为消息内容，这个测试可能旨在验证共享内存队列是否可以处理空消息或最小有效载荷。后面一个传输字符"a",也可完成测试。

```

postgres=# SELECT test_shm_mq(32768, (select string_agg(chr(32+(random()
)*95)::int), '') from generate_series(1,(100+900*random())::int)), 10
000, 1);
                                     test_shm_
mq
-----
(1 row)

```

测试共享内存队列处理较大消息的能力。输入消息大小字节为32768字节，过SQL查询动态生成一个随机长度和内容的字符串作为消息，这可以测试共享内存队列在面对不确定大小和内容的消息时的行为表现。相较前两个测试，这个等待时间明显更长，可能测试了共享内存队列在长时间操作下的行为或稳定性。

```

postgres=# SELECT test_shm_mq(100, (select string_agg(chr(32+(random()
)*95)::int), '') from generate_series(1,(100+200*random())::int)), 1000
0, 1);
 test_shm_mq
-----
(1 row)

```

这个测试用例可能关注于共享内存队列处理较小消息的能力。输入消息大小字节为100字节，与第三个测试类似，不过这次生成的消息长度预期会更短，测试队列处理短随机消息的能力。

```

postgres=#
postgres=# SELECT test_shm_mq_pipeline(16384, (select string_agg(chr(
32+(random()*95)::int), '') from generate_series(1,270000)), 200, 3);
 test_shm_mq_pipeline
-----
(1 row)

```

测试或模拟“管线化”（pipelining）处理共享内存消息队列的情况。管线化是一种在计算机科学中常用的技术，可以在一个步骤的输出直接成为下一个步骤的输入时，同时进行多个步骤的处理。

通过上述案例，提供了对共享内存队列功能面的广泛检验，虽然没有直接的输出，但通过这些函数调用的成功执行和内部逻辑，可以间接验证共享内存消息队列的稳定性和可靠性。